Contexte et objectifs de la phase 2

Après avoir travaillé sur le Data Storytelling pendant la phase 1, nous nous sommes intéressées aux outils qui pourraient nous permettre de transformer nos futures analyses en une application interactive.
Le sujet du projet demande de découvrir Dash, de comprendre comment construire une application simple sur une seule page, puis d'utiliser ces connaissances pour développer une application multipage.

Pour garder une progression logique, cette phase reste centrée sur le fonctionnement des outils.
Le jeu de données FAO, choisi pour Dazzle Dash, sera étudié dans la phase suivante.
Les chiffres de notre exemple sont donc fictifs et ne constituent pas des résultats d'analyse.

Objectifs

- Identifier les rôles respectifs de Plotly et de Dash.
- Comprendre la structure d'une application Dash : initialisation, layout et callbacks.
- Préparer une application monopage simple avec un filtre et un graphique interactif.
- Comprendre comment installer, lancer et vérifier l'application en local.
- Repérer les principes utiles pour l'application multipage finale.

Comprendre Plotly et Dash

Plotly : représenter les données

Plotly est une bibliothèque de visualisation qui permet de produire des graphiques interactifs en Python.
Plotly Express, accessible avec l'import **import plotly.express as px**, propose des fonctions simples pour créer des courbes, des histogrammes, des barres, des nuages de points ou des cartes.
Une figure obtenue peut ensuite être intégrée dans une application Dash.

Pour le projet FAO, Plotly pourrait par exemple aider à comparer des quantités alimentaires entre pays, à observer des évolutions historiques ou à représenter des répartitions géographiques.
Le choix d'un graphique dépendra toutefois des colonnes réellement présentes dans notre dataset.

Dash : construire une application

Dash est un framework open source pour développer des applications web interactives principalement en Python.
Il sert à organiser une interface contenant des titres, des textes, des graphiques et des contrôles, et à définir les réactions aux actions de l'utilisateur.
Le navigateur affiche l'interface ; l'application Python peut effectuer les traitements nécessaires.

Plotly	Dash
Crée des figures et visualisations	Organise l'interface de l'application
Permet le survol et le zoom sur les graphiques	Relie filtres, graphiques et autres composants
Produit une figure réutilisable	Affiche cette figure avec dcc.Graph

On retient que Plotly et Dash sont complémentaires : Plotly produit principalement les graphiques, tandis que Dash permet de les présenter et de les rendre interactifs à l'échelle de l'application.

Architecture d'une application Dash

Pour comprendre Dash, on a retenu trois notions : la création de l'application, son layout et ses callbacks.
Elles sont suffisamment distinctes pour être étudiées séparément, mais fonctionnent ensemble.

Initialiser l'application

L'instruction `app = Dash(__name__)` crée l'objet principal de l'application.
Cet objet sert ensuite à définir la page et à lancer le serveur de développement.
Pour notre premier exemple, un seul fichier **app.py** suffit.

Construire le layout

Le layout correspond à tout ce qui apparaît visuellement dans l'application : titres, paragraphes, listes, menus et graphiques.
Dash propose des composants HTML via **dash.html** et des composants interactifs via **dash.dcc**.
Ils sont placés les uns dans les autres pour organiser la page.

Faire fonctionner les callbacks

Un callback est une fonction Python reliée aux propriétés des composants.
Lorsqu'une propriété déclarée comme **Input change**, Dash déclenche la fonction ; sa valeur de retour est affectée à une propriété déclarée comme **Output**.
Ainsi, un menu déroulant peut commander le graphique affiché.

Schéma du fonctionnement :

sélection dans le menu → Input du callback → filtrage des données → nouvelle figure Plotly → Output du graphique.

Les composants et les callbacks

Élément	Rôle dans notre exemple
html.Div	Regrouper les composants de la page
html.H1 et html.P	Afficher un titre et une explication
dcc.Dropdown	Choisir le type d'usage alimentaire
dcc.Graph	Afficher une figure Plotly
Input	Définir la propriété dont la modification déclenche le callback
Output	Définir la propriété que le callback doit mettre à jour
id	Identifier un composant de manière unique

Le rôle de « value », « figure » et « id »

Dans un menu déroulant, la propriété value contient la sélection actuelle.
Dans un composant **dcc.Graph**, la propriété figure contient le graphique à afficher.
L'identifiant **id** permet au callback de retrouver précisément chacun de ces composants.
Il est donc important de choisir des noms cohérents et uniques.

Un **Input** ne signifie pas obligatoirement une zone où l'on écrit du texte : dans Dash, il peut s'agir de la valeur d'un menu, d'un bouton ou d'un autre composant.
De même, l'**Output** peut mettre à jour un graphique, du texte ou une autre propriété.

Application monopage : exemple commenté

Pour notre exercice technique, on a préparé un exemple volontairement petit, avec des valeurs fictives.
L'application affiche un menu permettant de choisir "Humain" ou "Animal", puis une courbe correspondant à la catégorie choisie.
Cette étape sert uniquement à apprendre Dash : elle ne décrit pas la situation alimentaire réelle.

Code complet à placer dans app.py

```
from dash import Dash, html, dcc, Input, Output, callback
import pandas as pd
import plotly.express as px
# Données inventées pour l'apprentissage
df = pd.DataFrame({
  "Année": [2000, 2005, 2010, 2000, 2005, 2010],
  "Usage": ["Humain", "Humain", "Humain",
        "Animal", "Animal", "Animal"],
  "Quantité": [40, 45, 52, 20, 25, 31]
})
app = Dash(__name__)
app.layout = html.Div([
  html.H1("Dazzle Dash - Démonstration"),
  html.P("Données fictives : découverte de Dash"),
  dcc.Dropdown(
    id="usage-dropdown",
    options=["Humain", "Animal"],
    value="Humain",
    clearable=False
  ),
  dcc.Graph(id="graphique-alimentaire")
])
@callback(
  Output("graphique-alimentaire", "figure"),
  Input("usage-dropdown", "value")
)
def actualiser_graphique(usage):
  donnees_filtrees = df[df["Usage"] == usage]
  fig = px.line(
    donnees_filtrees,
    x="Année", y="Quantité", markers=True,
    title=f"Évolution fictive - usage {usage.lower()}"
  )
  return fig
if __name__ == "__main__":
  app.run(debug=True)
```

Explication du fonctionnement

Les premières lignes importent Dash, les composants nécessaires, pandas et Plotly Express.
Pandas sert à représenter les données sous forme de tableau.
La commande `Dash(__name__)` initialise ensuite l'application.

Le layout contient les éléments visibles dans le navigateur.
Le Dropdown possède une valeur initiale `Humain` et le composant **Graph** est prêt à recevoir une figure.
Tous deux disposent d'un identifiant unique.

Le décorateur `@callback` relie la valeur du menu à la propriété figure du graphique.
La fonction `actualiser_graphique()` sélectionne les lignes correspondant à l'usage choisi puis crée une courbe.
Le `return` renvoie cette figure à Dash.

Enfin, la condition `if __name__ == "__main__"` permet de lancer le serveur lorsque le fichier est exécuté directement.
Le mode debug aide à repérer les erreurs pendant le développement, mais il ne doit pas être utilisé tel quel pour un service public.

Installation, exécution et vérifications

Préparer l'environnement

Il est préférable d'utiliser un environnement virtuel Python pour isoler les dépendances du projet.
Voici les commandes de base :

```
python -m venv .venv
# Sous Windows :
.venv\Scripts\activate
# Sous macOS / Linux :
source .venv/bin/activate
python -m pip install dash pandas plotly
```

Les commandes d'activation sont différentes selon le système d'exploitation : il faut utiliser uniquement celle qui correspond à son ordinateur.

Lancer et observer

```
python app.py
```

Lorsqu'il démarre avec sa configuration locale habituelle, le serveur Dash peut être consulté dans le navigateur à l'adresse **http://127.0.0.1:8050/**.
On pourra alors essayer de modifier l'option du menu pour observer le changement de courbe.

Points à vérifier

- La page s'affiche avec son titre, son menu et son graphique.
- L'option **Humain** est sélectionnée au démarrage.
- Choisir **Animal** met à jour le titre et les valeurs de la courbe.
- Le terminal ne présente pas d'erreur Python liée aux imports ou au callback.

À ce stade, le code est documenté et prêt à être testé.

Erreurs fréquentes

Situation	Piste de vérification
ModuleNotFoundError	Vérifier l'environnement Python et les paquets installés
Le graphique ne change pas	Vérifier les id, les propriétés Input/Output et la valeur retournée
Le navigateur ne trouve pas la page	Vérifier que le serveur est lancé et l'adresse affichée
Les valeurs sont incohérentes	Vérifier le type des colonnes et le filtrage pandas

Préparer la future application multipage

Une fois cette première application comprise et testée, le projet demandera une application multipage.
Cela ne signifie pas uniquement ajouter davantage de graphiques : il faudra répartir l'information de façon cohérente pour que l'utilisateur comprenne le récit sans se perdre.

La documentation actuelle de Dash recommande notamment le système **Dash Pages**.
Avec cette approche, le fichier principal utilise `Dash(__name__, use_pages=True)`, les pages sont placées dans un dossier pages/, chaque page est enregistrée avec `dash.register_page(__name__)` et `dash.page_container` accueille leur contenu.
Il existe aussi des architectures manuelles reposant sur **dcc.Location** et des **callbacks de navigation**.

Le sujet scolaire cite des noms de scripts comme `app.py`, `index.py`, `layout.py` et `callbacks.py`.
Ce sont des repères à conserver dans nos livrables.
Au moment de la conception, nous vérifierons quelle organisation répond le mieux aux attentes du sujet et aux recommandations actuelles de Dash, sans imposer immédiatement une structure définitive.

Une organisation possible plus tard

```
dazzle-dash/
 app.py
 pages/
   accueil.py
   evolutions.py
   comparaisons.py
 assets/
   style.css
 data/
 notebooks/
 docs/
 README.md
```

Cette arborescence est une proposition de travail et non un livrable déjà créé. Les pages définitives dépendront des résultats obtenus à partir des données FAO.

Bonnes pratiques pour Dazzle Dash

Favoriser la compréhension du public

Le directeur marketing décrit dans le sujet n'a pas besoin de voir toutes les explorations techniques. On devra sélectionner des indicateurs utiles, écrire des titres explicites, préciser les unités et accompagner les graphiques de courts messages qui expliquent ce qu'il faut retenir.
Les filtres devront aider à explorer les résultats sans masquer le récit principal.

Concevoir une interface lisible

Pour la présentation, Dash accepte des feuilles CSS placées notamment dans le dossier assets/.
Il est également possible d'utiliser Dash Bootstrap Components pour disposer d'une mise en page et de composants adaptés aux écrans.
On privilégiera des couleurs cohérentes, une hiérarchie de titres claire, des légendes compréhensibles et des contrastes suffisants.

Garder des analyses fiables

Une interface agréable ne suffit pas si les données sont mal interprétées.
Il faudra notamment vérifier les valeurs manquantes, les unités, les périodes, les catégories "nourriture humaine" et "alimentation animale", ainsi que les limites du jeu de données.
On fera attention à ne pas confondre disponibilité alimentaire, production et consommation réelle lorsque les variables ne permettent pas de mesurer ces notions de la même manière.

Sécurité, traçabilité et conformité

Le projet utilise des sources publiques, mais cela ne dispense pas de vérifier leur origine, les conditions de réutilisation et les éventuelles données personnelles.
On conservera les références des sources et on n'exposera pas d'informations sensibles dans le code ou dans un dépôt GitHub public.
Ces réflexes répondent aussi aux compétences de collecte et de traitement sécurisé mentionnées dans le sujet.

Synthèse et sources

Ce qu'on retient

Cette veille nous permis de comprendre que Dash n'est pas simplement une bibliothèque de graphiques : c'est un outil de construction d'applications interactives. Plotly sert à créer les visualisations, le layout organise les composants et les callbacks définissent leurs interactions.
On dispose maintenant d'un exemple monopage documenté, qui constituera une base pour la suite du projet.

La prochaine grande étape sera l'étude du dataset FAO : récupération des fichiers, inspection des colonnes, compréhension du périmètre et premières vérifications de qualité.
Le développement complet du dashboard viendra après l'analyse exploratoire.

Sources consultées

- Dash in 20 Minutes : https://dash.plotly.com/tutorial
- Dash Basic Callbacks : https://dash.plotly.com/basic-callbacks
- Dash Multi-Page Apps and URL Support : https://dash.plotly.com/urls
- Plotly Express in Python : https://plotly.com/python/plotly-express/
- Dash Bootstrap Components Documentation : https://www.dash-bootstrap-components.com/